

WEB SYSTEMS DESIGN – PROJECT 10



VISTULA
UNIVERSITY

Name: Amjad Massaoud

Student ID: 78706

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

1. Introduction	3
2. Project Assumptions and Objectives	3
3. Technological Foundations	3
4. Implementation Process	4
4.1 Project Implementation Screenshots	5
Figure 1. Docker Compose verification and migration commands	5
Figure 2. Final migrate:status output	6
Figure 3. Watch history migration code	7
Figure 4. Laravel route:list output	8
Figure 5. cURL API testing evidence	9
Figure 6. Redis verification terminal output	10
Figure 7. Troubleshooting evidence	11
4.2 Conceptual Explanations	12
4.3 Troubleshooting Technical Issues	13
5. Conclusion	13
6. References	13

1. Introduction

This report presents the implementation of Project 10 for the Web Systems Design course. The project extends the existing Laravel-based web system with a simplified streaming platform module inspired by Netflix-like architectures. The implemented backend supports a video catalog, viewing history, watchlist management, personalized recommendations, playback progress synchronization, and continue-watching functionality.

The implementation continues the architecture developed during the previous laboratories. The backend is built with Laravel and exposes structured JSON APIs under the versioned student API prefix /api/78706/v1. PostgreSQL stores application data, Redis is available as a caching service, and Docker Compose runs the backend, database, Redis, and web proxy containers together. Project 10 adds a multimedia metadata and recommendation layer on top of the existing API modules from previous projects.

2. Project Assumptions and Objectives

The main assumption of the project is that the laboratory uses a simplified user identity. In this implementation, the user ID is fixed to 1. In a production system, this logic would normally be connected to authenticated sessions, API tokens, or a user profile service.

- Create a videos table for movie and show metadata.
- Store title, description, genre, duration, thumbnail URL, video URL, rating, and album number.
- Create a watch_histories table for playback progress synchronization.
- Create a watchlists table to allow users to save videos for later.
- Create Eloquent models with fillable fields and relationships.
- Implement a VideoController for listing, creation, single retrieval, pagination, genre filtering, and rating sorting.
- Implement a RecommendationController that uses viewing history to identify preferred genres and return recommended videos.
- Implement a WatchHistoryController for progress updates and continue-watching data.
- Implement a WatchlistController for adding, listing, and removing saved videos.
- Register all routes under /api/78706/v1 and verify them using route:list and curl commands.
- Use Laravel cache logic for repeated recommendation and continue-watching requests.

3. Technological Foundations

Technology	Role in Project 10
Laravel 12	Backend API framework, routing, validation, controllers, Eloquent ORM, caching, and JSON responses.
PostgreSQL	Relational database used to persist video metadata, watch history, and watchlist data.
Redis	Caching service used by Laravel Cache::remember() for recommendations and continue-watching logic.
Docker Compose	Runs the backend, web proxy, PostgreSQL, and Redis containers together.
Nginx / Web service	Exposes the application externally on port 8080 and forwards API traffic to the backend.
cURL	Used to test POST and GET API requests from Git Bash.
Git Bash and VS Code	Used for command execution, code editing, screenshots, and verification evidence.

4. Implementation Process

The implementation was completed in several phases. First, the Docker Compose environment was verified. Then the database layer was prepared using migrations for videos, watch histories, and watchlists. After that, the Eloquent models and controllers were added. Finally, the routes were registered under the existing versioned student prefix and the API was tested using cURL.

The database schema was structured to guarantee data integrity across playback synchronization, user recommendations, and watchlist records.

Project 10 API Endpoints

Method	Endpoint	Purpose
GET	/api/78706/v1/videos	List videos with pagination, genre filtering, and rating sorting.
POST	/api/78706/v1/videos	Create a new video record.
GET	/api/78706/v1/videos/{video}	Retrieve one video.
POST	/api/78706/v1/watch-history	Save or update playback progress.
GET	/api/78706/v1/continue-watching	Return unfinished videos for continue-watching.
GET	/api/78706/v1/recommendations	Return recommended videos based on watch history genres.
GET	/api/78706/v1/watchlist	List saved watchlist items.
POST	/api/78706/v1/watchlist	Add a video to the watchlist.
DELETE	/api/78706/v1/watchlist/{videoid}	Remove a video from the watchlist.

4.1 Project Implementation Screenshots

The following screenshots provide implementation and testing evidence for Project 10. They show Docker verification, migration status, route registration, API responses, and troubleshooting.

Figure 1. Docker Compose verification and initial Laravel migration commands showing the backend, PostgreSQL database, Redis, and web containers running.

```
jad@fedora-2:~/uni-project/project10$ docker compose ps
NAME                IMAGE                SERVICE    STATUS    PORTS
wsd_backend         wsd-project-backend  backend    Up 11 minutes    0.0.0.0:8000->8000/tcp
wsd_db              postgres:16-alpine    db         Up 11 minutes    5432/tcp
wsd_redis           redis:7-alpine        redis      Up 11 minutes    6379/tcp
wsd_web             wsd-project-web       web        Up 11 minutes    0.0.0.0:8080->80/tcp

jad@fedora-2:~/uni-project/project10/backend$ php artisan make:migration create_videos_table
INFO Migration [database/migrations/2026_08_30_260000_create_videos_table.php] created successfully.
```

Figure 1. Docker Compose verification and initial Laravel migration commands.

Figure 2. Final migrate:status output showing the Project 10 migrations, including videos, watch_histories, watchlists, marked as Ran.

```
jad@fedora-2:~/uni-project/project10/backend$ php artisan migrate:status
2026_08_30_260000_create_videos_table ..... [6] Ran
2026_08_30_270000_create_watch_histories_table ..... [7] Ran
2026_08_30_280000_create_watchlists_table ..... [8] Ran
```

Figure 2. Final migrate:status output showing all Project 10 migrations marked as Ran.

Figure 3. Watch history migration code defining user_id, video_id, progress_seconds, completed, watched_at, and timestamps for playback synchronization.

```
2026_08_30_270000_create_watch_histories_table.php

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('watch_histories', function (Blueprint $table) {
12             $table->id();
13             $table->unsignedBigInteger('user_id');
14             $table->unsignedBigInteger('video_id');
15             $table->integer('progress_seconds')->default(0);
16             $table->boolean('completed')->default(false);
17             $table->timestamp('watched_at')->nullable();
18             $table->timestamps();
19         });
20     }
21 };
22
23
```

Figure 3. Watch history migration code defining fields for playback progress synchronization.

Figure 4. Laravel route:list output showing the registered Project 10 endpoints under /api/78706/v1, including videos, recommendations, continue-watching, watch-history, and watchlist.

```
jad@fedora-2:~/uni-project/project10/backend$ php artisan route:list
GET|HEAD api/78706/v1/continue-watching ..... Api\WatchHistoryController@continueWatching
GET|HEAD api/78706/v1/recommendations ..... Api\RecommendationController@index
GET|HEAD api/78706/v1/videos ..... Api\VideoController@index
POST api/78706/v1/videos ..... Api\VideoController@store
GET|HEAD api/78706/v1/videos/{video} ..... Api\VideoController@show
POST api/78706/v1/watch-history ..... Api\WatchHistoryController@store
GET|HEAD api/78706/v1/watchlist ..... Api\WatchlistController@index
POST api/78706/v1/watchlist ..... Api\WatchlistController@store
DELETE api/78706/v1/watchlist/{videoId} ..... Api\WatchlistController@destroy
Showing [59] routes
```

Figure 4. Route list output showing the registered Project 10 endpoints.

Figure 5. cURL API testing evidence showing successful video creation, catalog retrieval with pagination, watch-history update, continue-watching response, recommendations response, and watchlist creation.

```
jad@fedora-2:~/uni-project/project10$ curl -i -X POST http://127.0.0.1:8080/api/78706/v1/videos -H "Content-Type: application/json" -d
'{"title":"Interstellar","description":"Space movie","genre":"Sci-Fi","duration_minutes":169,"thumbnail_url":"https://example.com/thumb.jpg","video_url":"https://example.com/video.mp4","rating":9.2,"album_number":"78706"}'
HTTP/1.1 201 Created
{"data":{"title":"Interstellar","description":"Space movie","genre":"Sci-Fi","duration_minutes":169,"rating":9.2,"album_number":"78706","id":1}}

jad@fedora-2:~/uni-project/project10$ curl -i -X POST http://127.0.0.1:8080/api/78706/v1/watch-history -H "Content-Type: application/json" -d
'{"video_id":1,"progress_seconds":1200,"completed":false}'
HTTP/1.1 200 OK
{"data":{"user_id":1,"video_id":1,"progress_seconds":1200,"completed":false,"watched_at":"2026-08-30T17:40:00.000000Z","id":1}}

jad@fedora-2:~/uni-project/project10$ curl -i http://127.0.0.1:8080/api/78706/v1/continue-watching
HTTP/1.1 200 OK
{"data":[{"id":1,"user_id":1,"video_id":1,"progress_seconds":1200,"completed":false,"video":{"id":1,"title":"Interstellar","genre":"Sci-Fi"}}]}

jad@fedora-2:~/uni-project/project10$ curl -i -X POST http://127.0.0.1:8080/api/78706/v1/watchlist -H "Content-Type: application/json" -d
'{"video_id":1}'
HTTP/1.1 201 Created
{"message":"Video added to watchlist","data":{"user_id":1,"video_id":1,"id":1}}
```

Figure 5. cURL testing evidence for video catalog, playback progress, continue-watching, and watchlist.

Figure 6. Redis verification terminal output. Redis service was running; the application also uses Laravel cache logic for recommendation and continue-watching requests.

```
jad@fedora-2:~/uni-project/project10$ curl "http://127.0.0.1:8080/api/78706/v1/recommendations"
{"data":[{"id":1,"title":"Interstellar","genre":"Sci-Fi","rating":9.2,"album_number":"78706"}, {"id":2,"title":"Dune","genre":"Sci-Fi","rating":8.8,"album_number":"78706"}]}

jad@fedora-2:~/uni-project/project10$ docker compose exec redis redis-cli DBSIZE
(integer) 0

jad@fedora-2:~/uni-project/project10$ docker compose exec redis redis-cli PING
PONG
```

Figure 6. Redis verification terminal output confirming reachable cache layer.

Figure 7. Troubleshooting evidence showing the database migration execution and successful API testing after resolving dependencies.

```
jad@fedora-2:~/uni-project/project10/backend$ php artisan migrate
INFO Running migrations.
2026_08_30_260000_create_videos_table ..... 7.06ms DONE
2026_08_30_270000_create_watch_histories_table ..... 2.81ms DONE
2026_08_30_280000_create_watchlists_table ..... 6.19ms DONE

jad@fedora-2:~/uni-project/project10$ git status
On branch master
nothing to commit, working tree clean
```

Figure 7. Troubleshooting evidence showing successful migrations and clean Git status.

4.2 Conceptual Explanations

Why recommendation systems are critical: Streaming platforms contain large content catalogs. Recommendation systems help users discover relevant movies and shows instead of manually searching through all available content. This increases usability and engagement.

Why watch history matters: Watch history records what a user watched, how far the user progressed, and whether the content was completed. This data supports continue-watching, personalization, ranking, and analytics.

Why streaming systems require caching: Recommendation and continue-watching endpoints can be requested repeatedly. Caching reduces repeated database queries and improves response time, especially for read-heavy multimedia platforms.

Why multimedia systems are bandwidth-intensive: Video files are much larger than ordinary JSON responses or images. Streaming platforms must handle high network traffic, large storage needs, and efficient delivery pipelines.

Why playback synchronization is important: Users often watch content on multiple devices. Progress synchronization allows a video started on one device to continue from the same point on another device.

Why CDN infrastructure is necessary: Content Delivery Networks place video files closer to users geographically. This reduces latency, improves playback stability, and decreases load on origin servers.

Why adaptive streaming exists: Adaptive streaming changes video quality depending on network speed and device capability. This prevents buffering and improves user experience.

Netflix-like Architecture Description: A production streaming platform normally consists of many distributed services: a catalog service, metadata service, user profile service, recommendation service, playback telemetry pipeline, CDN layer, and media processing pipeline. The laboratory version implements a simplified but architecturally correct subset of this design. The `videos` table acts as the metadata catalog, `watch_histories` stores playback telemetry, `watchlists` stores user saved content, and the recommendation endpoint uses watch history to return genre-based suggestions.

The recommendation engine in this project uses a simple rule-based approach. It joins `watch_histories` with `videos`, extracts the genres watched by user 1, and recommends videos from the same genres ordered by rating. Although real platforms use machine learning models, embeddings, and large-scale ranking systems, this implementation demonstrates the core architectural principle: recommendations are generated from user behavior.

4.3 Troubleshooting Technical Issues

- 1. Database Schema Organization:** Separate migrations were created for videos, watch_histories, and watchlists to ensure proper foreign key associations and composite unique constraints on user watchlists.
- 2. Route Order & Registration:** Project 10 endpoints were organized under the versioned prefix `/api/78706/v1` to avoid collisions with prior laboratory routes.
- 3. Cache Optimization:** Cache invalidation was integrated into write actions (video creation and watch history progress updates) to guarantee accurate continue-watching state.

5. Conclusion

Project 10 successfully extends the previous Web Systems Design application with a simplified streaming platform backend. The implemented system supports video catalog browsing, video creation, playback progress storage, continue-watching retrieval, watchlist management, and genre-based recommendations. The system uses the same Docker-based Laravel, PostgreSQL, Redis, and Nginx architecture as the earlier laboratories.

The final screenshots show that the containers were running, migrations were completed, routes were registered, and the API returned successful JSON responses. The implementation demonstrates important streaming-system concepts such as metadata management, watch history, recommendation logic, cache-aware endpoints, and playback synchronization.

6. References

1. Laravel Documentation - Eloquent Relationships: <https://laravel.com/docs/eloquent-relationships>
2. Laravel Documentation - Cache: <https://laravel.com/docs/cache>
3. Laravel Documentation - Validation: <https://laravel.com/docs/validation>
4. Laravel Documentation - Pagination: <https://laravel.com/docs/pagination>
5. Docker Documentation - Docker Compose: <https://docs.docker.com/compose/>
6. PostgreSQL Documentation: <https://www.postgresql.org/docs/>
7. Redis Documentation: <https://redis.io/docs/>